

A Guide to gRPC Request-based Load Balancing

When Kubernetes Load Balancing Is Not Enough



Dominik Táskai

Senior Software Engineer
Linkerd Ambassador



 @dtaskai

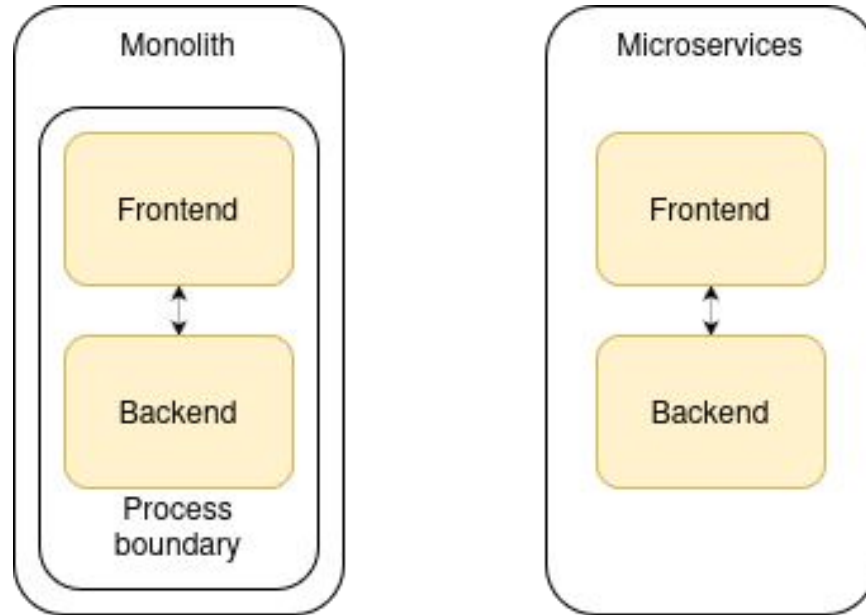
 @Dominik



The background is a complex, repeating pattern of triangles in various shades of blue and teal. The triangles are arranged in a way that creates a sense of depth and movement, with some triangles appearing to point towards the viewer and others away. The overall effect is a modern, geometric aesthetic.

Communication in Distributed Systems

Communication in Distributed Systems



Communication in Distributed Systems

→ REST (Representational State Transfer)

- ◆ Resource-oriented

→ RPC (Remote Procedure Call)

- ◆ Action-oriented

- ◆ Multiple flavors

 - JSON-RPC

 - HTTP-RPC

 - gRPC

→ GraphQL

→ WebSockets

Communication in Distributed Systems

REST

```
POST /cars HTTP/1.1
Host: api.example.com
Content-Type: application/json
```

```
{
  "make": "Toyota",
  "model": "Corolla",
  "year": 2023,
  "color": "Blue",
  "price": 20000
}
```

RPC

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "verify_transaction",
  "params": {
    "tx_id": "252162512"
  }
}
```

Communication in Distributed Systems

- Serialization format
 - ◆ But binary!
 - Improved performance
- Defined using proto files
 - ◆ Describes data structures and services
 - ◆ Simplifies development
- Language- and platform-neutral
- Serialization framework of gRPC



Communication in Distributed Systems

```
syntax = "proto3";

package transaction.v1;

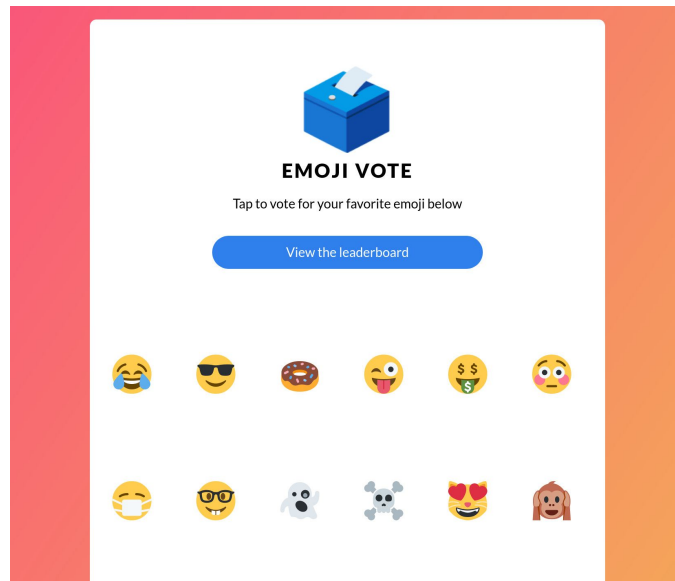
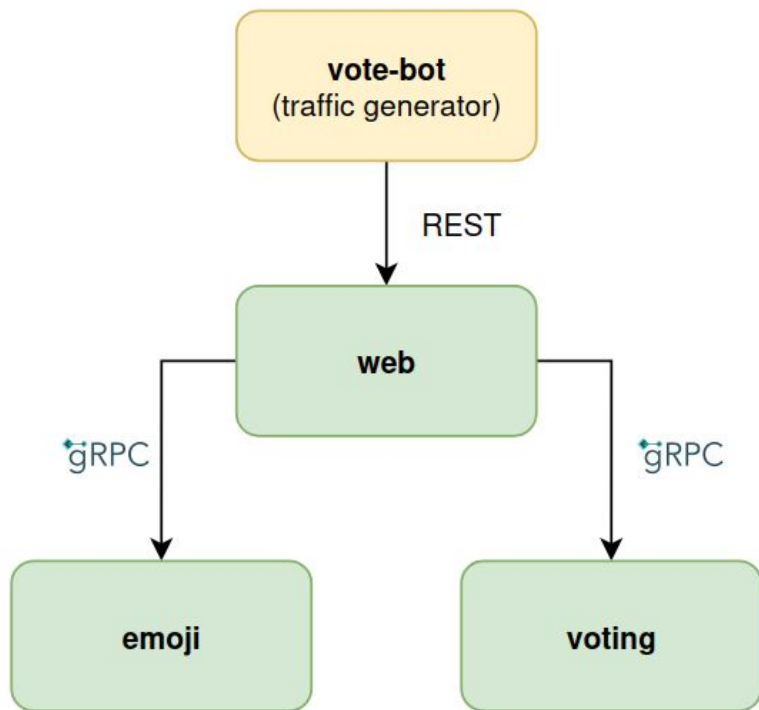
service TransactionVerifier {
  rpc VerifyTransaction(VerifyTransactionRequest) returns (VerifyTransactionResponse);
}

message VerifyTransactionRequest {
  string tx_id = 1;
}

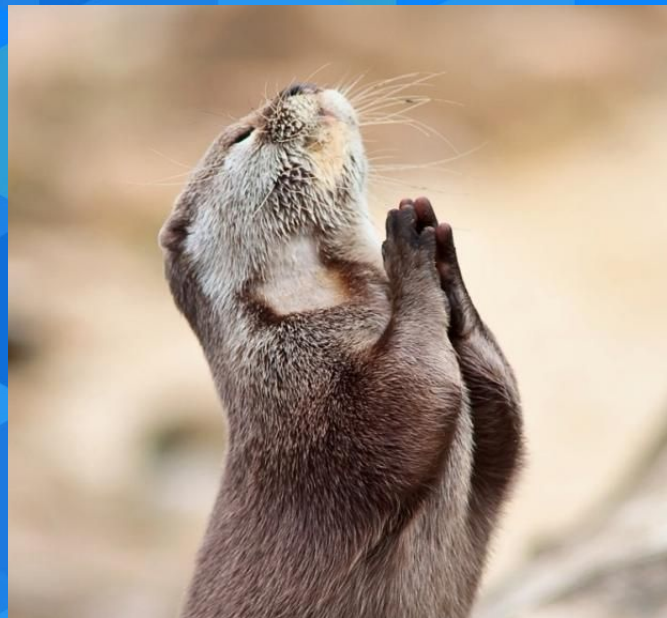
message VerifyTransactionResponse {
  enum TransactionStatus {
    UNKNOWN = 0;
    CONFIRMED = 1;
    PENDING = 2;
    FAILED = 3;
  }

  TransactionStatus status = 1;
  uint64 block_height = 2;
  string timestamp = 3;
}
```

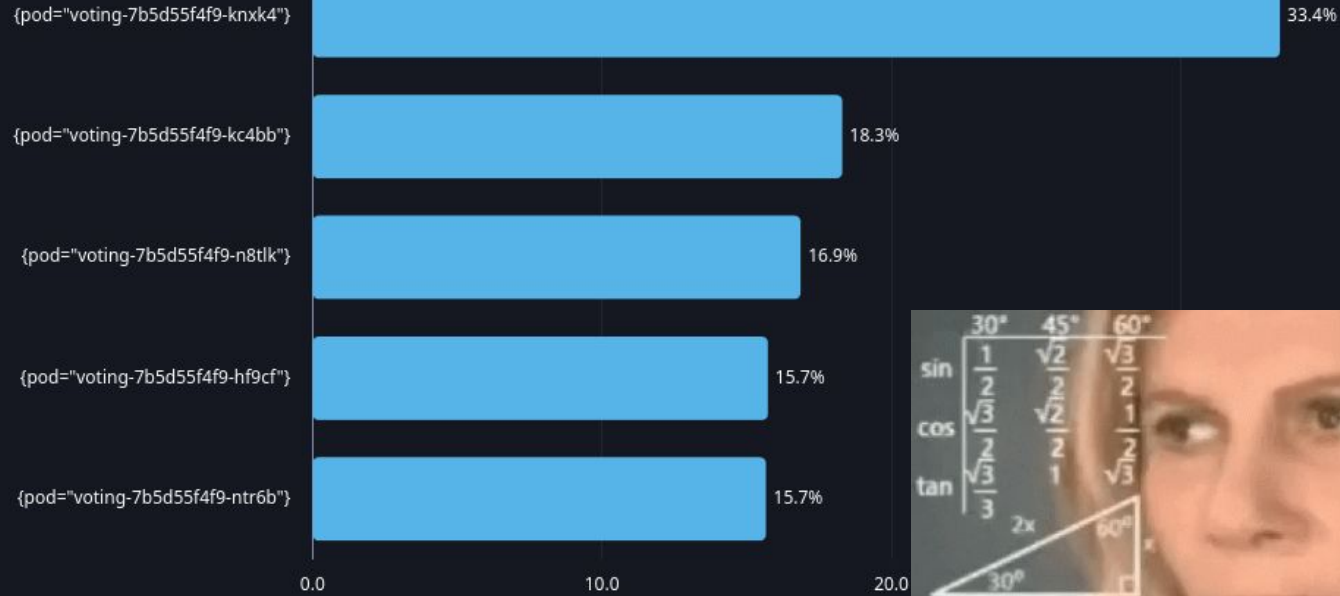

Communication in Distributed Systems



Demo Time



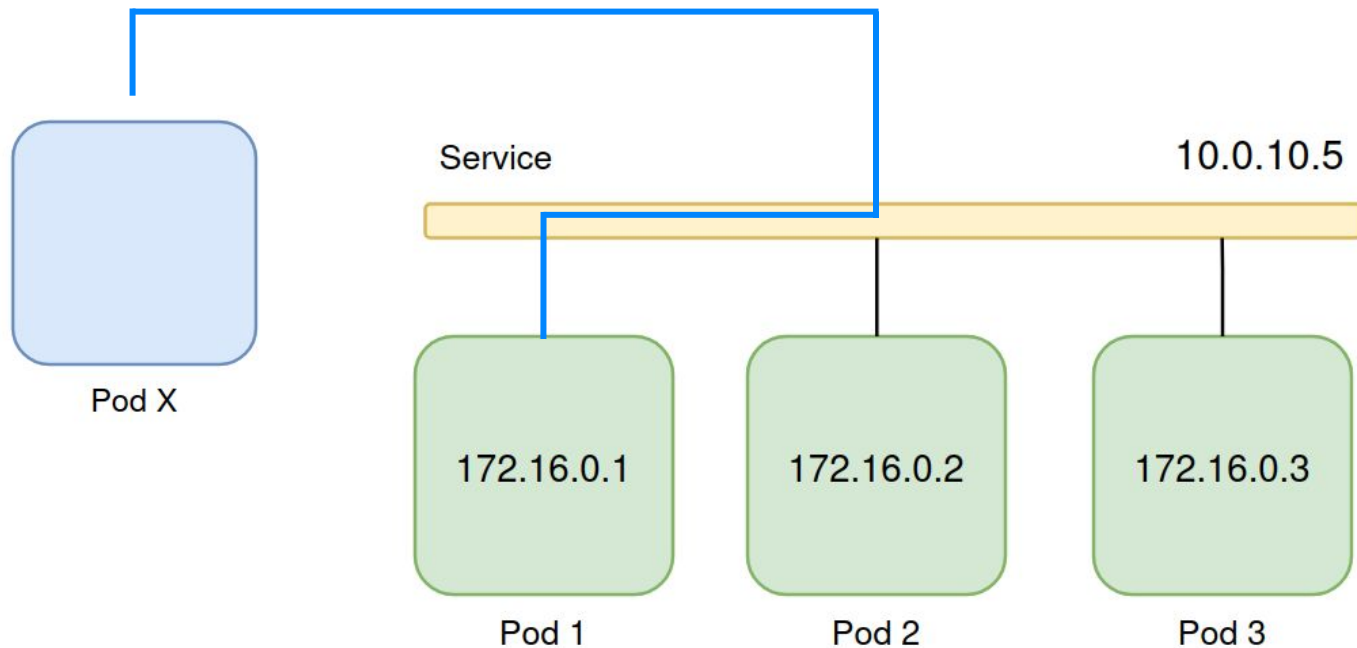
CPU usage



The background of the slide is a solid blue color overlaid with a complex geometric pattern of triangles in various shades of blue and teal. The triangles are arranged in a way that creates a sense of depth and movement, with some pointing towards the viewer and others receding.

How Kubernetes Load Balances Requests

How Kubernetes Load Balances Requests

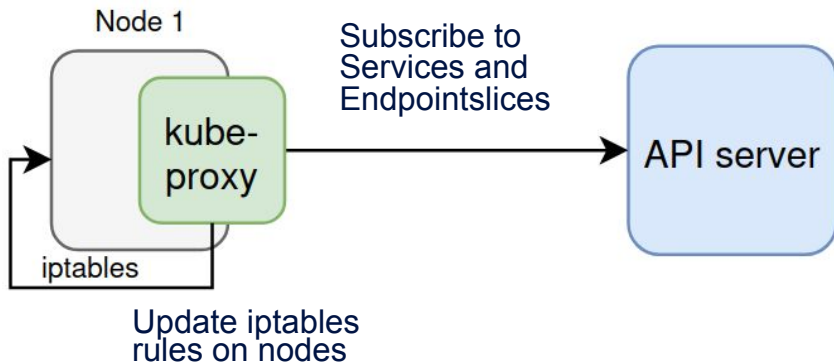


How Kubernetes Load Balances Requests

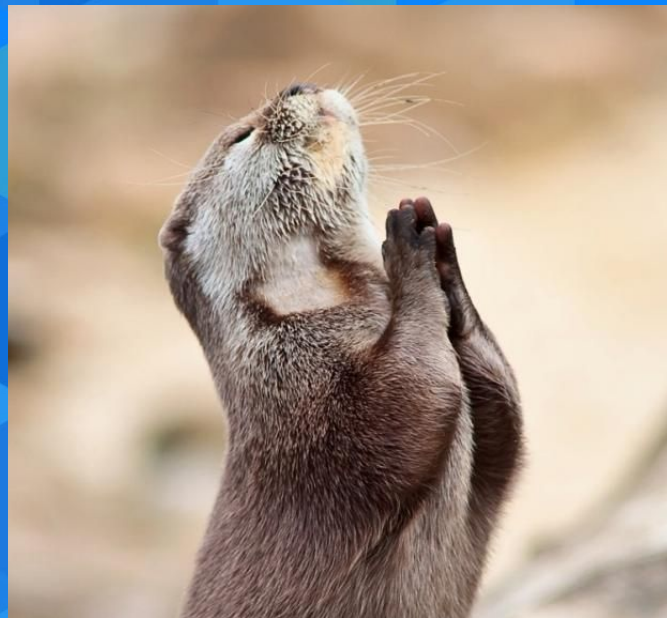


How Kubernetes Load Balances Requests

- Kubernetes Services don't exist
 - ◆ They never did...
- IPs are allocated by the control plane, stored in etcd



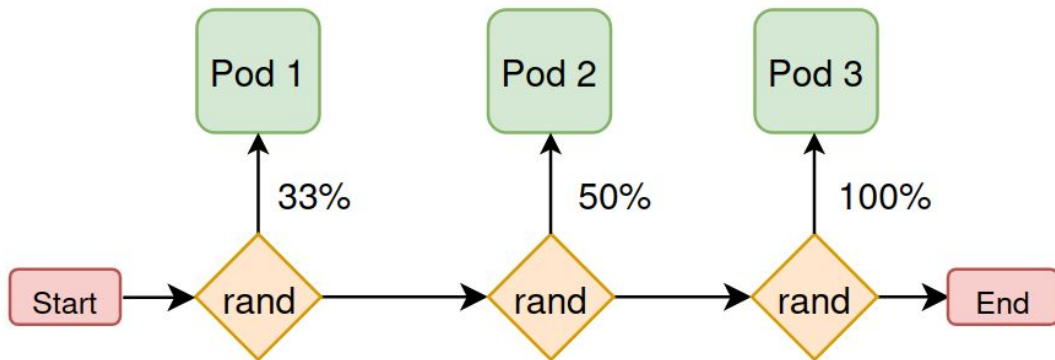
Demo Time



How Kubernetes Load Balances Requests

Chain KUBE-SVC-MSNDX3NF2CVUIS7P (1 references)

num	target	prot	opt	source	destination	
1	KUBE-MARK-MASQ	tcp	--	!10.244.0.0/16	10.96.209.84	/* emoji voto/voting-svc:grpc cluster IP */
2	KUBE-SEP-VFMTM06KZKFLAGMS	all	--	anywhere	anywhere	/* emoji voto/voting-svc:grpc -> 10.244.0.52:8080 */ statistic mode random probability 0.2000000019
3	KUBE-SEP-DEZ3FSQHENXW6EGU	all	--	anywhere	anywhere	/* emoji voto/voting-svc:grpc -> 10.244.0.53:8080 */ statistic mode random probability 0.2500000000
4	KUBE-SEP-40XDKPPIK4RXC7ET	all	--	anywhere	anywhere	/* emoji voto/voting-svc:grpc -> 10.244.0.54:8080 */ statistic mode random probability 0.3333333349
5	KUBE-SEP-5IIB5ZPKEKG0JMCV	all	--	anywhere	anywhere	/* emoji voto/voting-svc:grpc -> 10.244.0.55:8080 */ statistic mode random probability 0.5000000000
6	KUBE-SEP-S2FIORON3RKFRH7W	all	--	anywhere	anywhere	/* emoji voto/voting-svc:grpc -> 10.244.0.64:8080 */

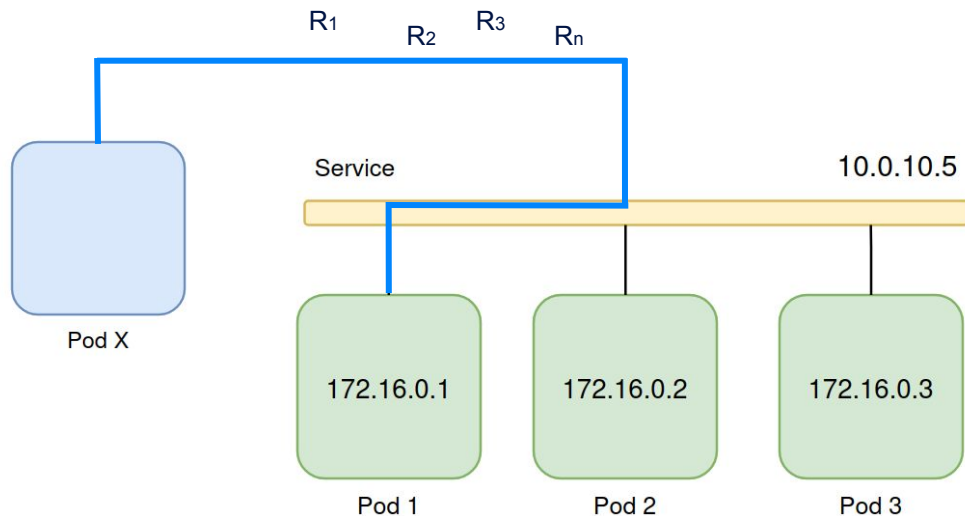


The background is a solid blue color overlaid with a complex pattern of overlapping triangles in various shades of blue, creating a 3D effect.

Why Is My Load Balancer Not Load Balancing

Why Is My Load Balancer Not Load Balancing

- gRPC is built on HTTP2
 - ◆ Single long-lived connection
 - ◆ Multiplexed requests, 1:N connection-requests
- Connection-level load balancing is not enough...



Why Is My Load Balancer Not Load Balancing

→ HTTP/1.1

- ◆ Processes requests sequentially
- ◆ Good at connection recycling
- ◆ Long-lived connections expire after a while



Connection-level load
balancing is good enough



The background is a complex, repeating pattern of triangles in various shades of blue and teal. The triangles are arranged in a way that creates a sense of depth and movement, with some triangles appearing to point towards the viewer and others away. The colors range from a deep navy blue to a light sky blue, with some teal accents.

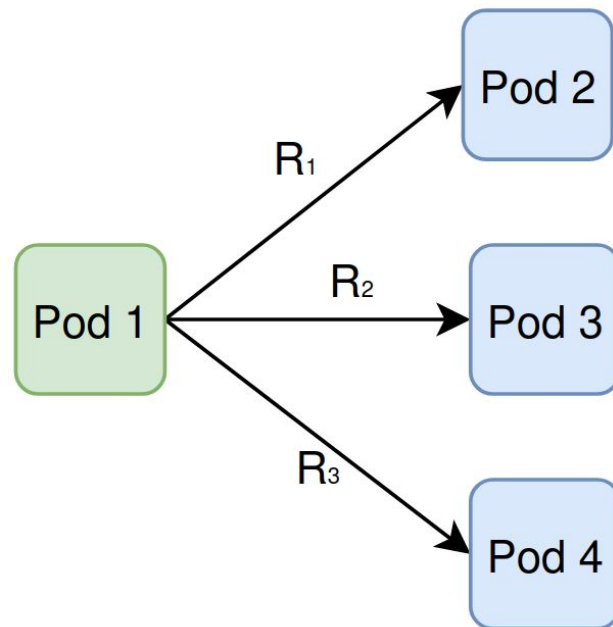
Request-level Load Balancing

Request-level Load Balancing

- Shift load balancing to L5/L7
- Connection to each destination
- Maintain connection pool (client-side)
 - ◆ Watch Kubernetes API
 - ◆ Update on changes



Implementation for each application **or...**





Ultralight, ultrafast, security-first
service mesh for Kubernetes.

- Created by Buoyant
- 8+ years in production
- 10,000+ Slack channel members
- 19,000+ GitHub stars
- 500+ contributors
- Weekly edge releases
- The first **CNCF graduated** mesh



Tradeshift



Clover



EVERQUOTE



GoSpotCheck



Ladbrokes



Mercedes-Benz



NORDSTROM

OfferUp



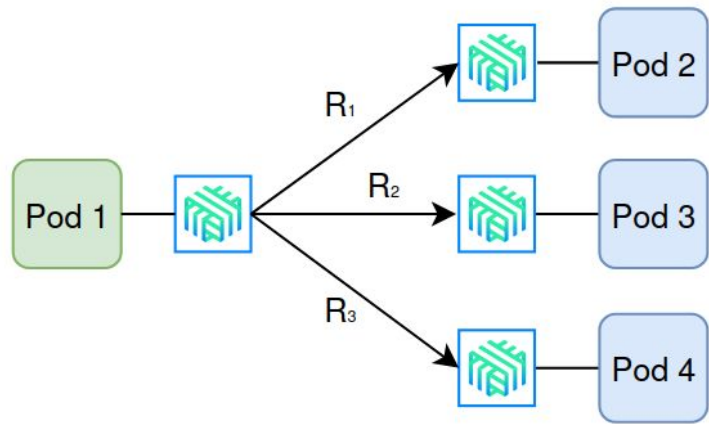
STRAVA

S U B S P A C E

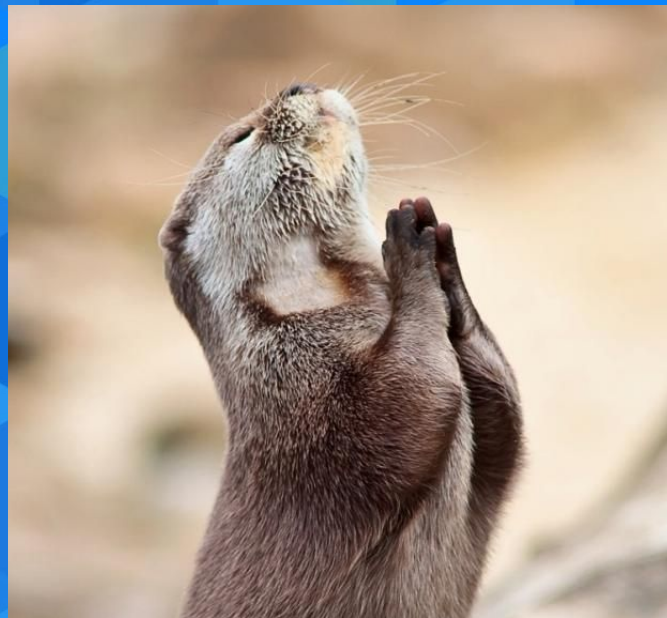


Request-level Load Balancing

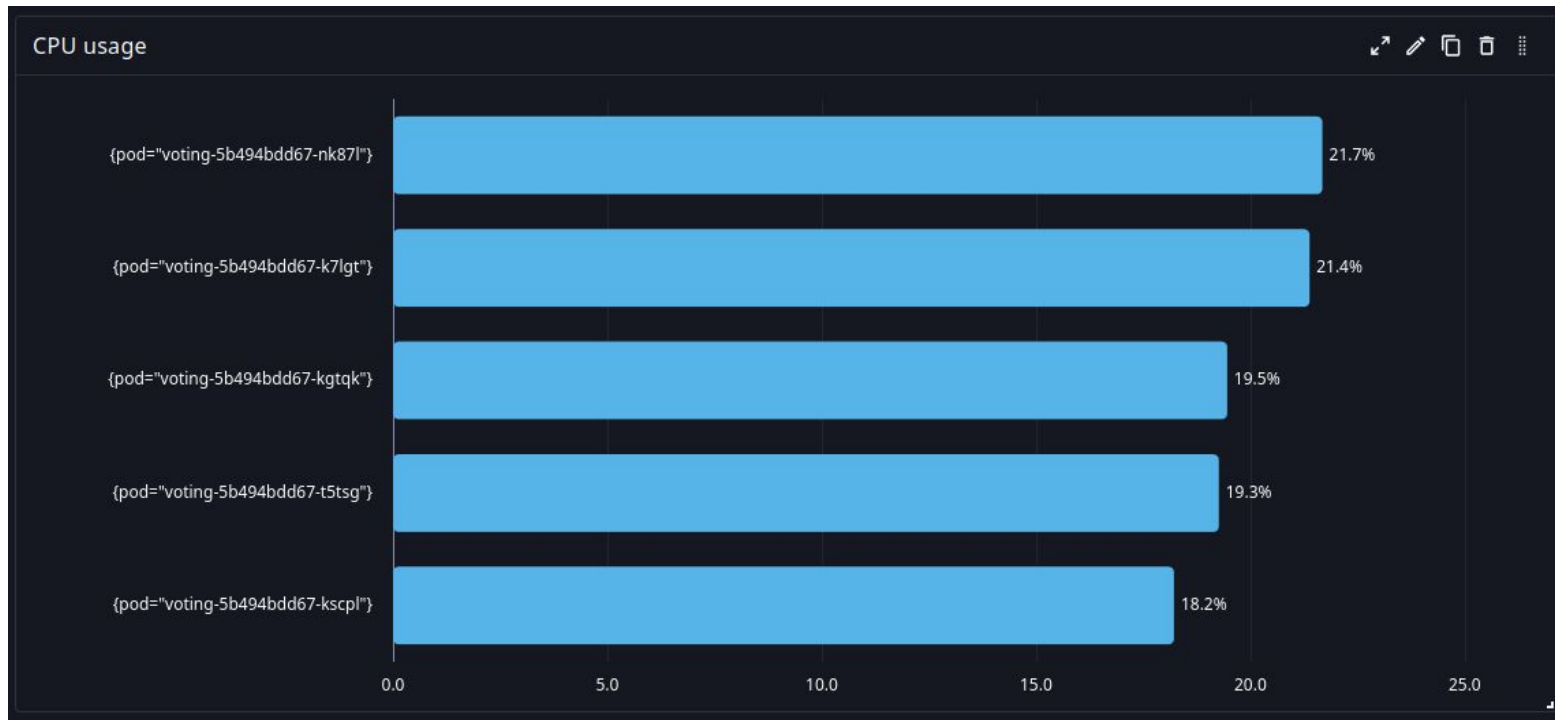
- Works with any
 - ◆ Language
 - ◆ gRPC client
 - ◆ Deployment model
- EWMA (exponentially-weighted moving average)
- Rust-based proxy 
- mTLS out-of-the-box



Demo Time



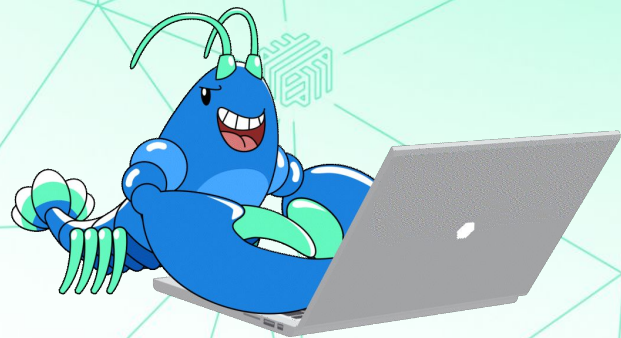
Request-level Load Balancing





Monthly live hands-on,
engineer-focused training from the
creators of the service mesh

SIGN UP TODAY!
buoyant.io/sma





SERVICE MESH
ACADEMY

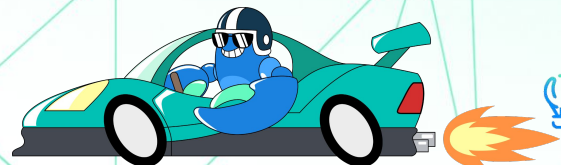
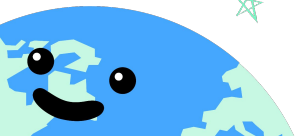


Get Service Mesh Certified!

Self-paced online courses.

With hands-on  LINKERD labs at
learn.buoyant.io

SIGN UP TODAY!
buoyant.io/sma



BUOYANT

Thank you

